

Reviewer Pack — Arithmetic Hierarchy Depth and DPLL Proof Path Length Inequa...

Entry c48cc28fb7a5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:47:51 UTC

Arithmetic Hierarchy Depth and DPLL Proof Path Length Inequality

Entry ID: c48cc28fb7a5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:47:51 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Hierarchy Theory **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every CNF formula φ with n variables, the depth of the arithmetic hierarchy required to express all instances of φ in natural numbers is linearly correlated with its DPLL proof path length, such that $A(\varphi) = \Theta(L(\varphi))$, where $A(\varphi)$ represents the arithmetic hierarchy and $L(\varphi)$ is a function of n .

Rationale (proposer's reasoning):

The arithmetic hierarchy captures the complexity of computations expressible by natural numbers. A direct correspondence between this hierarchy and DPLL proof path length could provide insights into the inherent structure of SAT problems and potentially lead to more efficient algorithms or lower bounds.

Taxonomy category: ArithmeticHierarchy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 311629976eb8a072

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of the CNF formulas (n variables) have a correlation coefficient between arithmetic hierarchy depth (A) and DPLL proof path length (L) greater than or equal to 0.8, with a p-value less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "arithmetic hierarchy" AND "DPLL proof complexity" - "CNF formula" AND depth AND DPLL AND satisfiability" - "hierarchy expression" IN natural numbers AND related TO DPLL proof path length

Top relevant hits considered: - [http://arxiv.org/abs/2101.11436v1] Challenges Encountered in Turkish Natural Language Processing Studies - [http://arxiv.org/abs/cs/0701184v2] Structure and Problem Hardness: Goal Asymmetry and DPLL Proofs in
 SAT-Based Planning - [http://arxiv.org/abs/2402.19127v2] Hankel Determinants of convoluted Catalan numbers and nonintersecting lattice paths: A bijective proof of Cigler's Conje

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
from itertools import combinations, product

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot
        max_row = i + matrix[i:].index(max(abs(row[i]) for row in matrix[i:]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for j in range(i + 1, rows):
            factor = Fraction(matrix[j][i], matrix[i][i])
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]

    return matrix

def is_singular(matrix):
    rows, cols = len(matrix), len(matrix[0])
    if rows != cols:
        return True
    identity = [[Fraction(1) if i == j else Fraction(0) for j in range(cols)] for i in range(rows)]
    reduced_matrix = gaussian_elimination([row[:] + col for row, col in zip(matrix, identity)])
    return any(row[i] != Fraction(0) for row, i in zip(reduced_matrix, range(cols)))

def dpll(cnf):
```

```

def dpll_helper(model, clause_index):
    if clause_index == len(cnf):
        return True
    literals = set(lit for clause in cnf[clause_index:] for lit in clause)
    for literal in literals:
        new_model = model.copy()
        if literal not in new_model and -literal not in new_model:
            new_model[literal] = True
            if dpll_helper(new_model, clause_index + 1):
                return True
        elif literal in new_model and new_model[literal]:
            continue
        else:
            new_model[literal] = False
            if dpll_helper(new_model, clause_index + 1):
                return True
    return False

return dpll_helper({}, 0)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        variables = list(range(1, n + 1))
        clauses = []
        for i in range(2**n):
            clause = [var if (i >> j) & 1 else -var for j, var in enumerate(variables)]
            clauses.append(clause)
        return clauses

    def arithmetic_hierarchy_depth(cnf):
        depth = 0
        for clause in cnf:
            depth = max(depth, len(set(abs(lit) for lit in clause)))
        return depth

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        cnf = generate_cnf(n)
        path_length = dpll(cnf)
        depth = arithmetic_hierarchy_depth(cnf)
        results.append({"n": n, "depth": depth, "path_length": path_length})

    correlation_coefficient = 0
    if len(results) > 1:
        x_mean = sum(result["depth"] for result in results) / len(results)
        y_mean = sum(result["path_length"] for result in results) / len(results)

        numerator = sum((result["depth"] - x_mean) * (result["path_length"] - y_mean) for result in results)
        denominator = sum((result["depth"] - x_mean)**2 * (result["path_length"] - y_mean)**2 for result in results)

        correlation_coefficient = numerator / denominator

```

```

return {
    "metric_name": "Arithmetic Hierarchy Depth vs DPLL Proof Path Length",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": "" if correlation_coefficient >= 0.8 else "Correlation coefficient < 0.8"
}

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))**0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient support for conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide a correlation coefficient and p-value for the CNF formulas. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14005
2	preregistration	ollama_remote	glm4:latest	0	0	9803
3	novelty	ollama_remote	glm4:latest	0	0	8601
4	novelty	ollama_remote	glm4:latest	0	0	10120
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16015
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10281
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10402
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13619
9	judge	ollama_remote	glm4:latest	0	0	11687

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104533 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c48cc28fb7a5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c48cc28fb7a5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c48cc28fb7a5.tar.gz` (if generated)